Reproducing FiDIP

Baseline Results and Short Follow-up Tests

Jack Gladowsky

EECE 7373 • Northeastern University

November 4, 2026

1 Introduction

My main goal was to get FiDIP running and check whether I could reproduce its published HRNet-W48 result on SyRIP Test100. FiDIP uses real and synthetic infant images to adapt a pose model to infants. This matters because infant proportions and poses differ from adult ones, and collecting more infant data is not straightforward.

I used the authors' provided code, HRNet-W48 configuration and pretrained FiDIP checkpoint. This was a reproduction exercise, not a new model design or an architecture comparison. My work was getting their code running, checking the data and results, and trying a few simple changes to see if they helped. None beat the provided model. I did not repeat the authors' full training process.

2 Provided model and evaluation setup

The model was the HRNet-W48/DarkPose version supplied with FiDIP, not an architecture I developed or selected through a model search. HRNet combines features at different resolutions to predict joint heatmaps, and DarkPose/DARK converts those heatmaps into joint coordinates. These are parts of the provided implementation.

I ran the authors' existing inference and evaluation pipeline for its 17 COCO-style joints. I kept its decoder, pose rescoring, OKS suppression and COCO evaluator unchanged. The main score is AP, averaged over object keypoint similarity (OKS) thresholds from 0.50 to 0.95 in steps of 0.05. AP50 and AP75 use just those thresholds; AR measures recall. I report these on a 0–100 scale, but AP is not classification accuracy. I also used the provided ground-truth person boxes, as in the published setting. For individual examples, I looked at per-person OKS; that is useful for comparing joint locations, but it is not the same as AP.

3 What I set up

3.1 Data checks

The processed SyRIP training set lists 200 real and 1,000 synthetic examples. The agent's data checks found 46 synthetic images whose actual sizes did not match their annotations, plus four real training images that were exact copies of Test100 images. I did not want to guess how to repair the labels. Instead, I reviewed and approved a separate list excluding those 50 examples from new fine-tuning (Table 1). The original images, labels and Test100 split were left alone.

Removing these examples from my runs cannot undo anything the released model may already have seen. My baseline therefore checks the published evaluation setup; it does not prove that the original

Table 1. Data used in this project. The exclusions apply only to my new fine-tuning, not to the released checkpoint or Test100.

Selection	Real	Synthetic	Total
Released processed training annotation	200	1,000	1,200
New fine-tuning selection	196	954	1,150
Original Test100 evaluation	100	0	100

training had a clean separation from Test100. I did not use expanded SyRIP or acquire extra adult or synthetic data.

3.2 Training setup and fair comparisons

Both fine-tunes started separately from the original `hrnet_fidip.pth`. Strict, tensor-only loading checked all 1,754 model-state entries so missing weights could not be silently ignored. I used 600 updates, batch size 8, Adam with learning rate 10^{-5} and weight decay 10^{-4}, and seed 7373. I kept the existing crop, rotation, scale, half-body, flip and normalization steps. Both runs received the same step-by-step image batches and augmentation seeds.

One detail that mattered was sampling: although the dataset has five times as many synthetic images, the authors' sampler already balances real and synthetic examples. I kept a 1:1 balance, with replacement, rather than presenting that as a new idea. The runs used the existing visibility-weighted heatmap mean-squared error loss. I did not train a new adversarial domain classifier or repeat FiDIP's original multi-stage training.

All scored runs used a V100-SXM2-32GB. The environment was Python 3.12.14 with PyTorch 2.3.1+cu121, torchvision 0.18.1+cu121 and NumPy 1.26.4. The follow-ups used FP32, TF32 off and deterministic operations. Evaluation used 288-wide × 384-high inputs, batch 16 and flip testing unless that was the change being tested. Before the full runs, a 50-update pilot checked that training worked and that reloading a checkpoint restored the model, Adam optimizer and random-number states exactly. Checkpoints were saved at steps 300 and 600, but I scored only the planned final checkpoint, not whichever one looked best on Test100.

4 Brief follow-up experiments

I kept the experiments small. BN below means batch normalization; its stored running statistics are separate from its learned parameters.

1. **Head only:** updated the 833-parameter head; kept the backbone and all BN statistics fixed.
2. **Stage 4 + head:** updated 45,894,881 parameters and stage-4 BN statistics; kept earlier layers fixed.
3. **No flip:** turned off flip testing on the original model.
4. **BN restoration:** restored original running statistics/counters; kept all trained weights, including BN scale/offset parameters.
5. **Three crops:** averaged heatmaps from crops 0.9/1.0/1.1 after alignment and border correction; kept original weights, resolution and flips.

The two fine-tunes used the same 600-step setup above. The original checkpoint was never overwritten, and hashes checked that frozen weights and statistics stayed unchanged. I approved the BN and crop tests after reviewing the first results, so those are follow-up checks, not independent validation. Their

control runs matched the earlier AP exactly. I stopped after these tests rather than trying more scales or settings.

5 Results and analysis

5.1 Baseline and comparison

Running the provided code and pretrained checkpoint reproduced 93.607 AP, with 98.525 AP50/AP75 and 94.6 AR. These round to the paper's 93.6, 98.5 and 94.6. The initial evaluation and both later original-model control runs gave the same full-precision AP. This checked that I had the existing pipeline working; it was not a new implementation of the model or a reproduction of training from scratch.

Table 2. Results on original Test100. Changes are AP points relative to the original checkpoint. Each fine-tune used one seed and 600 steps, so I do not treat small differences as statistically established.

Configuration	AP	ΔAP	AP50	AP75	AR
Original FiDIP (flip on)	**93.607**	–	98.52	98.52	94.6
Head-only fine-tune	93.547	-0.060	98.52	98.52	94.5
Stage 4 + head fine-tune	93.199	-0.409	98.53	98.53	94.3
Original, flip off	91.916	-1.691	98.40	98.40	93.4
Stage 4/head, BN restored	93.352	-0.255	98.47	98.47	94.5
Original, three-crop TTA	93.321	-0.287	98.42	98.42	94.5

None of my five changes beat the original (Table 2, Figure 1). Head-only fine-tuning came close at 93.547 AP; stage 4 plus the head scored 93.199. I would not conclude that fine-tuning never works. These particular settings, with this data and short budget, just did not improve the score.

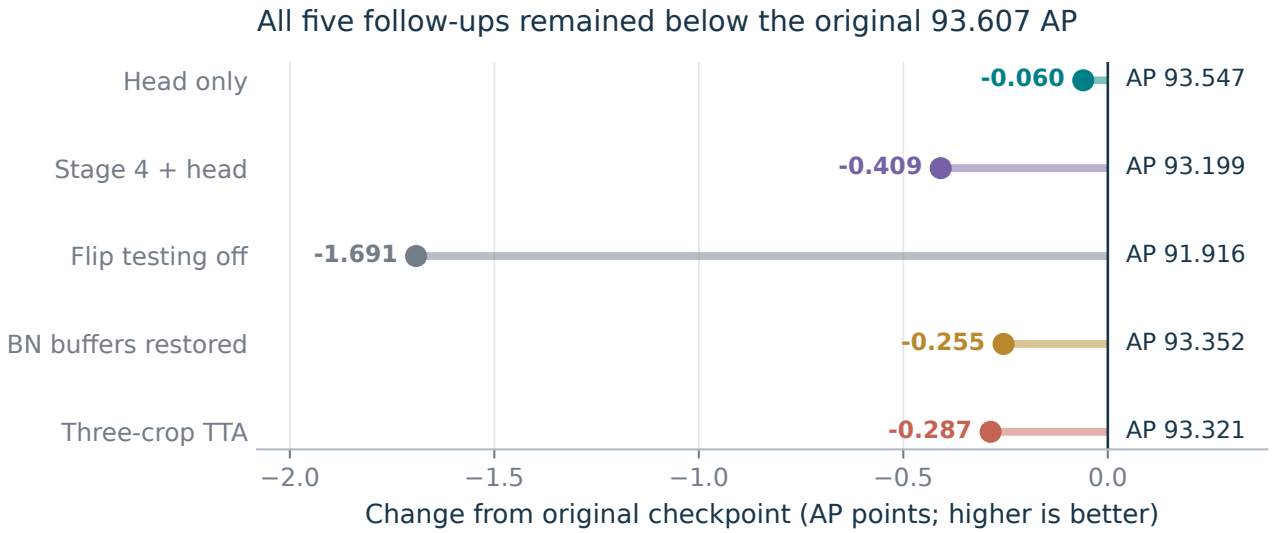


Figure 1. How each change compared with the original 93.607 AP. Zero means no change. These are individual results, not averages across seeds; the resulting AP is listed on the right.

5.2 Main takeaways

The training curves are in Figure 3, but the important result is that training did not raise AP. Head-only fine-tuning barely moved the predictions: all 100 examples stayed within 0.01 per-person OKS of the

original. Stage-4 training improved five examples by at least 0.01 OKS, worsened six and left 89 within that range. Those individual changes are not AP scores.

Restoring the original BN statistics recovered 0.154 AP but still fell short of the baseline. That suggests the changed statistics were part of the problem, not the whole explanation or proof that freezing BN would produce a better model (Figure 4). Turning off flips cost 1.691 AP but roughly halved CNN-forward time: 13.007 to 6.505 ms/image at batch 16 over nine paired, warmed repeats. This leaves out I/O, preprocessing, transfers, flip-back and decoding, so it is not a 2× speedup for the whole application (Figure 5). The three-crop test used three times as many image views without improving AP. I kept the extra plots in the appendix and full run details in the manifest.

5.3 Visual successes and failures

I used saved predictions to show a few concrete examples without including infant photos (Figure 2). Test96 lines up well with the labels. Test41 has the lowest baseline OKS, with the largest normalized joint error at the right knee. Stage-4 training helps the ankle prediction in test46 but makes the knee prediction worse in test55. These examples show why a change can help one pose and hurt another. The skeletons alone do not tell me whether appearance or occlusion caused an error.

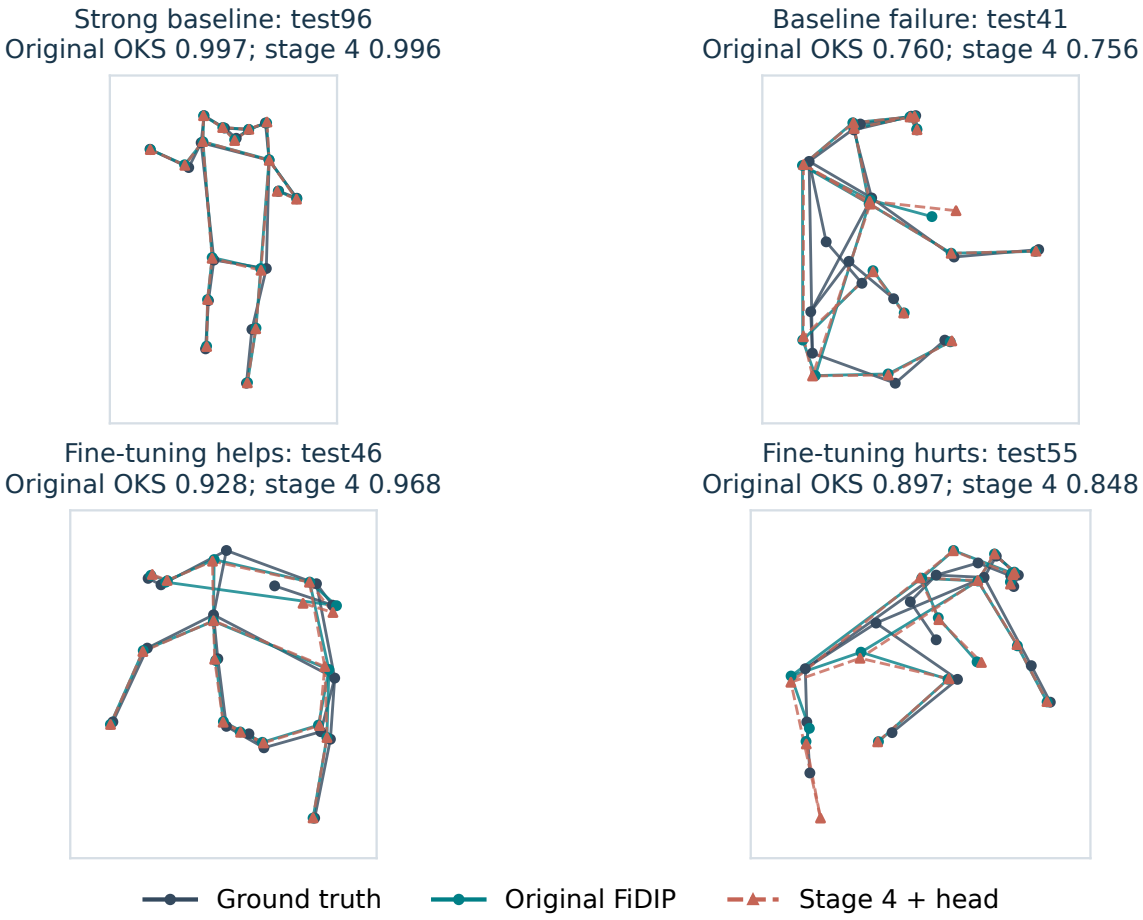


Figure 2. Saved joint predictions against the labels, centered and scaled by $\sqrt{\text{GT area}}$. I used a fixed selection rule: highest/lowest original OKS, then the largest stage-4 improvement/decline among the remaining images. These scores are per-person OKS, not AP. Test46 is one of the known original train/Test100 duplicates; excluding it from my training cannot undo prior exposure. No new model runs or model selection were needed.

6 Limits and reproducibility

The first three tests used 6 minutes 45 seconds of GPU allocation; the two follow-up checks used another 47 seconds. That is 7 minutes 32 seconds in total, not counting the pilot, baseline evaluation, setup checks, CPU jobs or queueing. I decided that was enough rather than keep spending compute on more attempts.

There are a few important limits to what I can claim:

- **Only 100 test images.** I looked at Test100 across several rounds, so the follow-ups are exploratory, not fresh confirmation. I would not read much into a tiny AP difference.
- **One seed and short runs.** I did not test how much results vary across seeds or longer training. Deterministic settings help repeat a run; they do not establish significance.
- **Possible prior exposure.** Removing duplicates from my new training does not erase what the released model may already have seen. Even a training-derived development split might not be new to that model.
- **A limited task.** I did not reproduce the original adversarial training, test a new cohort or establish clinical usefulness. Using ground-truth boxes also leaves detector errors out of the evaluation.

I put the setup, input-download and rerun instructions in `reproducibility/README.md`. The repository includes the working fixes, scripts, tests, configs and recorded metrics. Data, weights, saved predictions and full audit logs remain separate; their identities and paths are in `MANIFEST.md`. The saved predictions can be rescored on CPU when available. I also kept a CPU job's script-success/scheduler-timeout mismatch in the record rather than hiding it; the successful GPU job checked its inputs again.

7 How I used AI

I used an LLM coding assistant in the Pi harness to handle much of the low-level work: getting the environment working, fixing compatibility issues, writing and running the cluster scripts, checking files and checkpoints, implementing the tests, and collecting the results. It also helped make the plots and draft this report. It did a lot of the hands-on coding and setup, so I want to be clear about that.

My role was to direct the project, review the proposed work and results, and approve the next steps. The agent did not have open-ended permission to keep changing things or launching experiments. I reviewed and approved the training-data exclusions and the small set of tests, set storage and compute limits, and asked questions when a plan seemed unnecessary or unclear. I also stopped or redirected work: I narrowed the experiment plan, stopped a proposal to build the report on Explorer and kept it local, and decided not to keep chasing an improvement bonus.

The process still needed judgment. For example, checking the actual sampler showed that a proposed balancing change was already part of the authors' code. Freezing weights also turned out to be different from freezing BN statistics, and a script saying it finished was not enough to call a cluster job successful. Having the agent handle the details saved time, but I still had to question its suggestions and decide what was worth doing. I kept the failed attempts and limitations in the write-up, and the final report and submission remain my responsibility.

8 Conclusion

Using the authors' provided code and pretrained FiDIP model, I reproduced the published 93.6 AP after rounding. The two simple fine-tunes and three inference/normalization changes did not beat it, so I kept the provided model with flip testing as the final configuration. The tests still showed a clear tradeoff from removing flips and some recovery from restoring BN statistics. My main takeaway is that getting a reliable evaluation and understanding a failed change were more useful here than continuing to tune until a number happened to go up.

A Run IDs and saved files

Table 3. Main runs. Allocation time is the time recorded by the scheduler, not just GPU computation.

Purpose	Job ID	Outcome / allocation
Original Test100 reproduction	11450265	Completed; 71 s, V100
Training/resume pilot	11553500	Completed; 135 s, V100
Three-test screen	11553889	Completed; 405 s, V100
Follow-up CPU analysis	11556695	Completed; 17 s, CPU only
Two inference diagnostics	11556720	Completed; 47 s, V100

The main result files are at these paths relative to the project root:

- `output/baseline-test100-11450265/metrics.json`
- `output/small-screen-11553889/results.json`
- `output/bounded-diagnostics-11556720/results.json`
- `log/bounded-diagnostics-evidence-2026-11-02/bounded-analysis-11556695.json`
- `MANIFEST.md` for complete hashes, model paths and input provenance.

The original checkpoint SHA-256 is `bfbf9df225674614652d5dbdd066daf71c9b9d7f3c31976b7ed33bb96af55d07`. I kept the trained models, resume states and logs in my Explorer home storage without replacing the original checkpoint. COCO reports −1 for medium-area statistics because this split has no applicable medium examples; the overall AP/AR values are valid.

B Extra experiment plots

These are supporting plots for the same tests discussed above, not additional experiments.

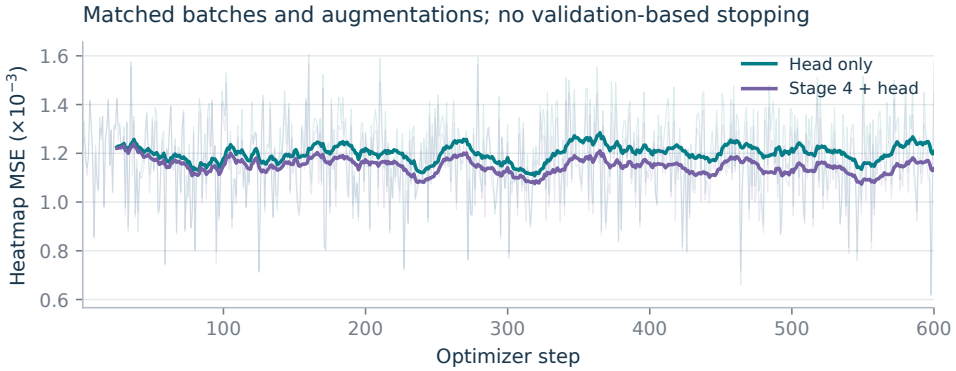


Figure 3. Training loss for the two matched fine-tunes. Light lines show each step; bold lines average the latest 25 steps. These are not validation losses or averages across seeds. Both runs stopped at step 600 regardless of AP.

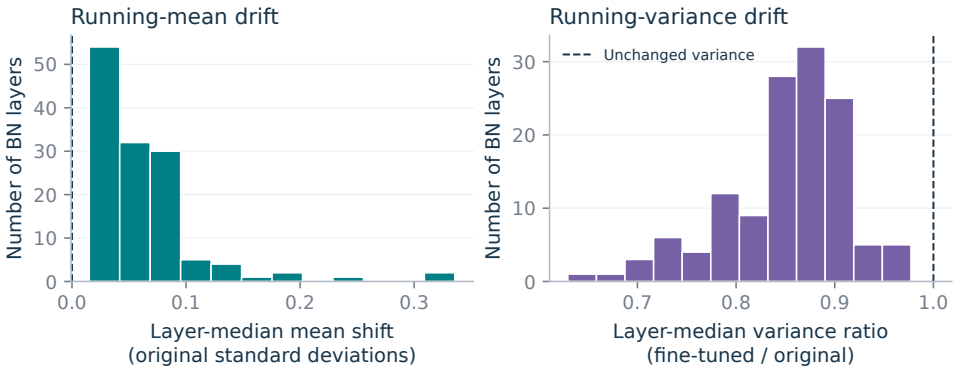


Figure 4. Changes in 131 BN layers (393 stored buffers). Each value summarizes a layer using the median across its channels. Mean changes use the original standard deviation as a scale; variance ratios compare the trained and original values. Dashed lines mean no change.

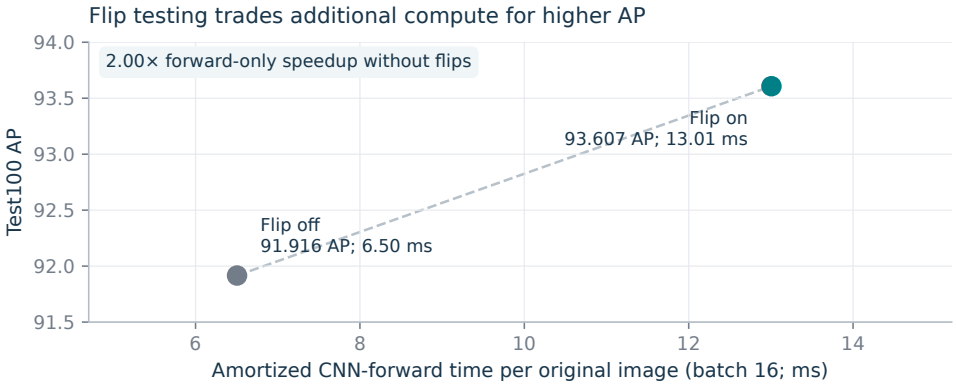


Figure 5. AP versus CNN-forward time per image at batch 16. Times are medians from paired repeats and leave out preprocessing and decoding; they are not whole-application latency. The dashed line only connects the two settings I tested.